

Service-Role → RLS Migration, Full-Product Plan

Status: design, not yet built. Drafted 2026-05-29. Supersedes the inline-filter-only tenancy model. Companion to [cross-org-reviews.md](#) (whose grant model this plan makes enforceable in the database).

Today **every** backend DB call uses the `service_role` key, which bypasses RLS. Tenant isolation is 100% application-layer convention; the ~50 RLS policies that exist never execute at runtime, and several are malformed (`auth.uid()` subqueries with no `GRANT SELECT` , `app_metadata` claims that disagree with the DB membership source of truth). This plan moves the boundary into Postgres.

0. JWT mode, VERIFIED 2026-05-29 (dev, read-only probe with controls). HS256 IS honoured; #1 and #2 both viable.

Method: GET `/rest/v1/studios?select=id&limit=1` with combinations of apikey + bearer. Results:

apikey	bearer	result	meaning
service (legacy JWT)	none	200 (rows)	service key = RLS bypass (today's app path)
publishable (<code>sb_publishable_</code>)	HS256 signed w/ <code>SUPABASE_JWT_SECRET</code> , <code>role=authenticated</code>	200 (empty, RLS-scoped)	HS256 token honoured + RLS applied
service	HS256 wrong secret	401 "No suitable key or wrong key type"	bearer signature IS verified; not ignored
(none)	HS256	401 "No API key found"	apikey is a mandatory gateway gate

The **wrong-secret 401 is the clincher**: a bad bearer fails the request even with a privileged apikey, so the correct-secret **200 proves the signature matched**. PostgREST verifies bearers against **both** the ES256 JWKS key **and** the legacy shared `SUPABASE_JWT_SECRET` (HS256). (An earlier note here said HS256 was rejected, that was a misread of a `401` at the bare `/rest/v1/` *root*, which only requires a secret-tier apikey; corrected by this controlled probe.)

Key formats on dev: `SUPABASE_ANON_KEY` = new `sb_publishable_...` ; `SUPABASE_SERVICE_ROLE_KEY` = **still a legacy JWT** (`eyJ...`). Mixed but both valid. JWKS also serves one **ES256/EC** key (real GoTrue user tokens).

Impact on the four identities, both request identities work as originally designed: - **Identity #1 (user), viable:** forward the **real GoTrue user token** (`CurrentUser.token` , ES256) as bearer + the **publishable** apikey. PostgREST applies RLS per its claims. (The `authenticated` HS256 probe returning a

200 empty set under the existing claim-policies is the expected RLS-applied shape, confirm with a real logged-in session token before cutover.) - **Identity #2 (system), original design viable TODAY:** mint a `role=arthound_system` HS256 token signed with `SUPABASE_JWT_SECRET` (+ publishable apikey). Once `arthound_system` exists and is granted to `authenticator`, PostgREST `SET ROLE`s to it. **Caveat (the reason to still evaluate option a):** this depends on the **legacy shared HS256 secret**, which Supabase is steering projects away from in favour of asymmetric **JWT Signing Keys**. If HS256 verification is ever disabled, the minted system token breaks. So the durable variant is **(a) register a custom JWT Signing Key we control** and mint the system token with it (PostgREST trusts it via JWKS), same design, future-proof signer. Decide (a-durable) vs (HS256-now) before building #2. (*Researching option (a) next.*) - **Identity #4 (service_role):** unchanged; it's the legacy-JWT secret key (full bypass).

Still TODO: re-run this probe against **prod** (`rhzlmka1wpmjufruacky`) for parity; confirm #1 with a real session token; choose the #2 signer (a vs HS256-now) and test end-to-end on dev before any migration.

0a. Identity #2 signer, option (a) researched (2026-05-29). Both paths viable; durable vs simple.

Because the app talks to the DB through **PostgREST (REST)**, not a direct pg connection, the only way to run as a non-`authenticated` Postgres role is a **JWT whose role claim names that role**, there is no per-request `SET ROLE` without one. So identity #2 needs a token signed by *something PostgREST trusts*. Two real signers (both require `arthound_system` created + `grant arthound_system to authenticator`):

- **HS256-now:** mint `role=arthound_system` HS256 over `SUPABASE_JWT_SECRET`. **Proven to work today** (\$0 probe). Simplest, zero new infra. **Risk:** couples #2 to the **legacy shared secret**, which the new signing-keys regime is steering projects away from; a future step that disables HS256 verification silently breaks every background job. Detectable (re-run the probe), but a latent landmine.
- **(a) Imported asymmetric signing key, durable upgrade path.** Supabase docs confirm import is supported (`supabase gen signing-key --algorithm ES256` generates a key you retain, imported as a **standby** key; not re-extractable after import). A **standby key is published in JWKS but signs nothing**, "*made available for discovery, but no JWT is signed with it yet.*" That is exactly the verify-only state we need: PostgREST verifies our `role=arthound_system` service tokens via JWKS while GoTrue keeps signing **user** tokens with the *current* key. We simply never click **Rotate**. **No dependency on the legacy HS256 secret** → immune to its deprecation. (Earlier worry that an imported key would have to be *current* and thus sign user tokens is resolved by the standby semantics, research 2026-05-29.) Supabase-generated keys don't help (private key not re-exposed); **import** is the enabling detail. Requires CLI ≥ ~v2.102 / dashboard (current dev CLI is v2.98.1).

DECISION (2026-05-29): HS256-now for v1 + a deprecation-watch; option (a) is the confirmed, non-breaking upgrade path.

- **Signer:** mint `role=arthound_system` HS256 tokens over `SUPABASE_JWT_SECRET` (+ publishable apikey). Proven working today (\$0 probe). Simplest, zero new key infra, **clean separation** (legacy secret signs only our service tokens; GoTrue's ES256 signs only user tokens, they never touch).

- **The deprecation-watch is concrete, not vague:** the legacy JWT secret is on Supabase's removal track with **late-2026** as the stated milestone (legacy keys "remain available... removed in late 2026"; rotating them is already disabled). Build a startup/CI check that **re-runs the \$0 HS256-acceptance probe** and alerts if it ever returns 401, that is the trigger to execute the upgrade.
- **Upgrade path to (a) is non-breaking (confirmed):** import our ES256 key as standby (dual-verification window, both signers verify simultaneously), switch the minter from HS256 to our key, done. No user impact, no flag-day. So choosing HS256-now does **not** corner us.
- **Rejected for v1:** option (a) up front (needs a CLI upgrade + live auth-config change to stand up now, for no v1 benefit over HS256); and **Supabase Third-Party Auth** (own external issuer/JWKS, cleanest separation but the most infra), note both as future options.
- Either signer still needs `arthound_system` created + `grant arthound_system to authenticator`, and a guard that our custom role isn't caught by Supabase's reserved-role protection. **Build + test #2 end-to-end on dev** (create role → grant → tiny system-only policy → mint token → confirm PostgREST `SET ROLE` s and a system read works) **before any tenancy migration**.

UPDATE (2026-06-05): option (a) EXECUTED on DEV (hardening item #1). HS256 → ES256 swap done. The deprecation-watch's upgrade path is now the live signer on dev. An ES256 P-256 key (`supabase gen signing-key --algorithm ES256, kid fb73f484-...`) was imported as a **standby** JWT signing key into the dev project (`kwrlqqnzcnpjqvesygxo`) JWKS — published for verification, signs nothing (never Rotated, so GoTrue keeps signing user tokens with the current key `2493ba8b-...`). The minter (`lib/system_auth.py:_mint()`) signs with it (private JWK in env `ARTHOUND_SYSTEM_SIGNING_JWK`, `kid` header) when the var is set, else falls back to the legacy HS256-over-`SUPABASE_JWT_SECRET` path (unset the var + restart = instant rollback; both verify during Supabase's dual-verification window). `aud` stays `authenticated` (changing it = \$0b#3, separate). Verified on dev: JWKS shows both kids; ES256 token accepted by PostgREST; `SET ROLE arthound_system` confirmed (system read returns rows, anon 0); live server boots clean under `USE_USER_IDENTITY=1` (proves the startup-probe gate passed on ES256); HS256 fallback still accepted. CLI-version worry was moot — `generate` works on v2.98.1; `import` is dashboard-side. **PROD DONE 2026-06-05 too** — separate prod key (kid `f456b5dd-...`) imported as standby into `rhzlmka1wpmjufruacky` JWKS (now carries `edde3e89-...` current + `f456b5dd-...` standby); code shipped `dev→main→prod` (prod `86ab7d4`); `ARTHOUND_SYSTEM_SIGNING_JWK` set as a Railway secret (raw single-line JWK, no quotes — Railway injects to env, no `.env` file in the image). Pre-deploy standalone probe confirmed prod PostgREST accepts the ES256 token + `SET ROLE` (3 source_credentials rows, anon 0); post-deploy prod serves healthy under `USE_USER_IDENTITY=1` (green deploy = the live ES256 startup gate passed). Rollback = delete the Railway var → redeploy → HS256. Cutover rule proven on both: **IMPORT KEY FIRST**, then activate the env var (the startup probe is a blocking go/no-go).

0d. Identity model, END-TO-END VALIDATED ON DEV 2026-05-29 (reversible probe, fully torn down)

A scoped probe (`_rls_probe` table + `_rls_probe_current_studio_ids()` SECURITY DEFINER fn + a temporary `arthound_system` NOLOGIN role granted to `authenticator`, two seed rows for studio A & B) was created on dev, exercised via PostgREST REST with five minted HS256 bearers, then **completely**

dropped (post-checks: role/fn/table/policies all 0; ledger untouched at 58 / 20260529000003). Results, **all five exactly as predicted:**

Persona	bearer	result
user A (member of studio A)	<code>role=authenticated, sub=<userA></code>	1 row, studioA only ✓
user B (member of studio B)	<code>role=authenticated, sub=<userB></code>	1 row, studioB only ✓
system	<code>role=arthound_system, sub=0...0</code>	2 rows, A+B ✓
spoof	<code>role=authenticated</code> + body <code>app_metadata.role=arthound_system</code>	0 rows ✓ (body claim grants nothing)
anon	publishable apikey, no bearer	0 rows (RLS denies; not 401, the apikey alone is <code>anon</code> , which has no policy)

This proves the load-bearing mechanics before we build anything: - Identity #1: a real user JWT → PostgREST applies RLS → user sees only their own org via the predicate fn. - Identity #2: a `role=arthound_system` HS256 token → PostgREST `SET ROLE` s to it → its `using(true)` system policy returns all rows. The signer (HS256 over `SUPABASE_JWT_SECRET`) and the role-claim → `SET ROLE` path both work. - **\$0b#2 anti-spoof CONFIRMED empirically:** a token with `app_metadata.role=arthound_system` in the body but `role=authenticated` at the JWT root is treated as `authenticated` and sees nothing, the body claim is not an authority. (Reinforces: system policies must key on `current_user='arthound_system'` , the actual SET ROLE, never a claim read.) - Tenant isolation holds symmetrically (A can't see B, B can't see A).

One refinement for the real build (from the anon result): a deny-all table returns **200 with 0 rows** to `anon` , not 401. Fine for tenant tables (empty = denied). For F-pattern system-only tables we still want truly no access, which `anon` already has (no policy = no rows), and the system role reaches them by its own policy. No change needed, just noted so "200 empty" isn't mistaken for a bug during testing.

The four runtime identities we are building toward:

#	Identity	Auth	RLS	Used by
1	User	anon key + user JWT	enforced	all <code>routes/*</code> request handlers
2	System	anon key + minted <code>arthound_system</code> JWT	enforced (narrow policies)	poll loop, sync-from-cron/webhook, trim, attachment-copy worker, token refresh
3	RPC (cross-tenant)	user JWT → <code>SECURITY DEFINER</code> fn	bypassed <i>inside</i> fn; in-fn authz is the guard	the 13 cross-org writes below
4	service_role	service key	bypassed	migrations + break-glass + two structural carve-outs (\$0c)

0b. Pre-build gates, STOP if any of these is unresolved

Three adversarial review passes (security, completeness, ops) surfaced load-bearing items that gate the whole migration. Resolve each before writing a migration file.

1. **Prod migration history, VERIFIED NOT ARMED 2026-05-29 (the feared blocker was a stale phantom).** A live read-only audit of both databases found the slot-demotion `DROP COLUMN s ( 20260528000002/3 )` are **already applied AND ledgered on both dev and prod**, columns gone, ledger rows present, no duplicate rows. Both DBs are at **parity: 58 ledger rows, max 20260529000003**, **ledger-vs-files diff empty both directions**. There is **no armed destructive migration**; the "🐛 STILL PENDING" note in the prod-backlog memory was stale (its own header already said "FULLY RESOLVED"). **No reconciliation was needed.** (An attempted "fix" during the audit fabricated a non-existent `20260529000004` migration and was fully reverted same-session, see the incident note in `[[project_prod_migration_backlog]]`; net DB change = zero.) - Standing rule for the RLS rollout regardless: **never blind db push to prod**, apply RLS migrations explicitly and re-verify the ledger-vs-files diff (CRLF-stripped) is empty before and after. CLI gotcha confirmed: `db query --linked` works via the Management API without `SUPABASE_DB_PASSWORD`; `db push / migration repair` need the direct pg connection. Always re-link back to dev after any prod-linked work, and sanity-check `current_database()` / `inet_server_addr()` before any prod write.
2. **is_system() must be a Postgres-role identity, never a JWT claim (CRITICAL).** System-table policies must check `current_user = 'arthound_system'` (PostgREST actually did `SET ROLE`, which only succeeds if the verified JWT's root `role` claim names a role `authenticator` is a member of). Do **not** check `request.jwt.claims->'role'` anywhere, PostgREST copies *all* JWT claims there, so a claim-read is spoofable by any validly-signed token carrying `role:arthound_system` in its body and would hand out the F-table god-policies. Add a test: a normal `authenticated` JWT with an injected top-level `role:arthound_system` claim (but no SET-ROLE grant) must be **denied** on `source_credentials`.
3. **lib/auth.py:_decode_jwt trusts the attacker-controlled alg header (CRITICAL, pre-existing).** `alg = header.get("alg", "HS256")` + HS256-verify-against-secret is the classic RS256→HS256 confusion vector, and it's now doubly dangerous because the system identity's entire trust rests on HS256 over `SUPABASE_JWT_SECRET`. **Pin algorithms** per environment (don't derive from the header). Verify the system token on a separate path with its own pinned alg and a **distinct aud** (e.g. `aud='arthound_system'`, configured in PostgREST) so a system token can never satisfy the user path's `aud='authenticated'` check, and explicitly reject any user-path token whose root role is `arthound_system` or whose `sub` is the system sentinel. Add the "RS256 user token re-signed as HS256 with the public key is rejected (401)" test.
4. **JWT mode (from \$0), the load-bearing unknown.** If prod is on asymmetric keys, the minted HS256 system token is rejected → falls back to `anon` → every background job silently dies (the loops swallow exceptions). Resolve with the \$0 smoke test before anything else.

0c. The two structural service-role carve-outs (must be named, not "cleaned up")

Two service-role uses are **structurally unavoidable** and must be explicitly sanctioned, the break-glass-unreachable test (§9) must allowlist exactly these and nothing else:

- **GoTrue Admin API** (`/auth/v1/admin/users/ ...`), used by `members.py:_get_user_emails` (every org-hub member render), `payload.py:get_outbox` (`_resolve_user`), `user.py` delete-account, and `auth.py` signup ×2. This is **not** a PostgreSQL table, RLS and the system role do not apply; only the service key reaches it. Either keep it as a named service-role consumer, or (preferred) denormalize email into a `profiles` table written at signup and read via RLS, eliminating most admin lookups. Org creation at signup (`studios / vendors` INSERT before any membership row exists) is the same carve-out, or a `create_org_with_owner` RPC (§6).
- **Storage blob I/O** (`lib/attachments.py:_storage_headers`), upload and proxied byte-serving stay service-role; the **user-context metadata gate** (§7) is what authorizes access before any byte is fetched. Do **not** migrate `_storage_headers`; allowlist it. A naive "remove all service-role" sweep here breaks all attachment viewing.

These are why identity #4's scope is "migrations + break-glass + **these two**," not "migrations + break-glass only."

1. The root-cause fix, predicate functions (identity-agnostic, hardened)

Every malformed policy and the JWT-claim-vs-DB-authority split have one root and one fix: resolve membership through `SECURITY DEFINER` helpers that read `studio_members / vendor_members` (the source the app already trusts), so `authenticated` never needs direct `SELECT` on membership tables.

Every function below is `SECURITY DEFINER`, `STABLE`, and `SET search_path = ''` (empty path → all names fully qualified → no schema-shadowing hijack). This is non-negotiable (security finding C5).

```
-- 00_predicates.sql
```

```

returns boolean language sql immutable set search_path = '' as $$
select array_position(array['view','comment','act'], p_have)
       ≥ array_position(array['view','comment','act'], p_need)
$$;

revoke all on function public.current_studio_ids, public.current_vendor_ids, public.is_my_or
public.is_org_admin, public.is_link_party, public.has_grant, public.perm_at_least from pub
-- EXECUTE to authenticated ONLY – the system role uses current_user='arthound_system' ($5),
-- these membership predicates, so granting them to arthound_system only widens the surface
-- "predicate EXECUTE to system"). Predicate-fn creation is locked down so a later migration
-- silently REPLACE is_my_org() with 'select true': REVOKE CREATE on schema public from non-
-- roles, and a CI catalog test asserts proscdef=true + proconfig pins search_path on each.
grant execute on function public.current_studio_ids, public.current_vendor_ids, public.is_my
public.is_org_admin, public.is_link_party, public.has_grant, public.perm_at_least
to authenticated;

```

Roles must exist before this migration. `arthound_system` and `arthound_rpc` are created as the very first statements of migration 1 (idempotent `do $$ ... if not exists ... create role ... $$`), so every later `grant` / `alter function owner` can reference them (ops-finding: grant-before-role ordering bug).

Wrapping `auth.uid()` as `(select auth.uid())` and each predicate as `in (select fn())` lets the planner hoist them to an **InitPlan** evaluated once per statement rather than once per row, the single most important RLS-perf move on `replicated_assets` (finding B1). Confirm with `EXPLAIN ANALYZE` before trusting it; fall back to a per-request transaction-local GUC if a per-row blowup appears.

1a. FORCE × SECURITY DEFINER, the recursion/silent-deny footgun (do not skip)

A `SECURITY DEFINER` helper reads its consulted table **as the function owner**. The standard reason `current_studio_ids()` doesn't recurse is the **table-owner exemption**: when the function owner is also the table owner and the table is *not* FORCE'd, that read skips RLS. §3's **"FORCE every tenant table"** removes exactly that exemption from the predicate-consulted tables, which arms two failure modes (not one):

1. **Recursion:** `select ... from generated_work → policy studio_id in (select current_studio_ids()) → helper reads studio_members → now FORCE'd, so owner is subject to studio_members' policy → which is studio_id in (select current_studio_ids()) → back to the helper → stack-depth error.`
2. **Silent-deny (the quiet one):** even if you rewrite the membership policy to a non-recursive `user_id = (select auth.uid())`, that policy is `TO authenticated` and the helper runs as the *owner* role, so **no policy applies to the owner under FORCE** → default-deny → the helper returns **empty**, and every B/C/D policy that calls it silently denies all rows. No error, just an app that shows nothing.

The only thing masking both in dev is the postgres owner's `rolbypassrles` (BYPASSRLS beats FORCE). That is a load-bearing, unstated dependency that breaks the instant the migration/predicate owner is a role without BYPASSRLS. **This generalizes to every DEFINER predicate that reads a FORCE'd table:** `is_link_party → studio_vendor_links, has_grant → review_grant`, not just membership.

Fix (removes the BYPASSRLS dependency entirely): keep the **predicate-consulted tables out of the FORCE set**, `studio_members`, `vendor_members`, `studio_vendor_links`, and (when the review subtree

ships) `review_grant`. They are then read via the table-owner exemption, which depends only on **function-owner == table-owner** (guaranteed when migrations run as one role) and *not* on the `rolbypassrls` attribute. These tables still have RLS **enabled** with policies, so the direct user query path (`authenticated`, a non-owner, RLS always applies to non-owners, FORCE or not) stays fully enforced; only the owner-run helper reads get the exemption. The data these tables hold is low sensitivity (who-is-in-which-org, which-orgs-are-linked); the sensitive payloads/rates/assets live in tables that stay FORCE'd.

Preflight-assert the invariant this rests on (`reowner` of each exempt table == `proowner` of the predicate fns), and add the §9 tests: (a) a brand-new **zero-membership** user calling `resolve_my_membership` returns empty with **no stack-depth error**; (b) a **FORCE-set regression test** asserting `studio_members` / `vendor_members` / `studio_vendor_links` are *never* in `pg_class.relforcerowsecurity` (so a future "FORCE everything" change can't silently re-arm this). If you ever *must* FORCE one of these, the alternative is owning the predicate fns by a BYPASSRLS role and asserting `rolbypassrls` on that owner in preflight, but FORCE-exemption is preferred because it removes the attribute dependency rather than documenting it.

2. Table classification, all ~50 tables, one pattern each

Six patterns; corrections from the adversarial reviews folded in. **F = no user policy at all (deny-all for users); the system role gets its own narrow policy.** Every table not listed as **F** also needs system-role policies where `system?` = yes.

A, Owner/org-record (read own org row)

Table	Read	Write	system?
<code>studios</code>	<code>id in (select current_studio_ids())</code>	RPC only (create/rename)	no
<code>vendors</code>	<code>id in (select current_vendor_ids())</code>	RPC only	no

B, Single-org tenant (`studio_id` / `vendor_id` in membership)

`studio_members`, `vendor_members`, `canonical_assets`, `workflow_steps`, `workflow_step_dependencies` (via FK chain), `estimate_config`, `payload_templates`, `vendor_studio_ingest_templates`, `generated_work` (+ `deleted_at` is null default), `scenario_sessions` (+ `scenario_messages/products/assets/work` via session FK). Membership-table **writes** are RPC-only (bootstrap deadlock, §7).

C, Polymorphic owner (`owner_type`, `owner_id`), bind the type (finding M3)

`replicated_assets`, `replicated_products`, `replicated_item_types`, `replicated_work`, `source_field_mappings`, `source_entity_definitions`, `sync_cursors`, `sync_log`, `source_schema_cache`, `init_jobs`, `field_bucket_override_log`, `schema_drift_events`, `asset_reviews`, `review_attachments` (via review FK). Read = `is_my_org(owner_type, owner_id)`.

D, ### D: Dual-party cross-org (party to the link), **split SELECT vs write** (finding C4)

`studio_vendor_links`, `studio_vendor_invites`, `payload_dispatches`, `payload_field_mappings`, `payload_access_log`, `link_cancellation_audit`, `link_cancellation_dispatches`, `estimate_share_series`, `estimate_share_dispatches`, `estimate_share_access_log`.
(`payload_export_records` is dual-party-read but **vendor-write-only**, see the cross-org READ hotspots below; `estimate_matrix` is **not** here, it is vendor-private in full, see §2 corrections.)

E, Grant-based (the review subtree, planned)

`review` (container), `review_step`, `review_comment`, `review_grant`, `review_event`, **plus the org-scoped definition layer** `review_status_def`, `review_workflow_def`, `review_step_def` (Pattern C, `is_my_org` owner, completeness review caught these were uncovered). Read = `is_my_org(owner_org...)` OR `has_grant('<subject>', id, 'view')`. - **Hard dependency on column shape:** `is_my_org` / `has_grant` need the **two-column** (`owner_org_type` text, `owner_org_id` uuid) form, but cross-org-reviews.md §3 currently writes a single `owner_org` / `author_org` value. Pin the two-column decomposition in that doc **before** any Pattern E policy or `has_grant` ships. **Do not ship `has_grant` or any Pattern E policy in the cutover migrations**, gate the entire grant subtree behind the cross-org-reviews feature migration that creates `review_grant`; a `has_grant` referencing a non-existent table is a landmine that deny-fails reads under FORCE RLS. Cutover predicate set is `current_*_ids`, `is_my_org`, `is_org_admin`, `is_link_party` only. - **Partner status-label resolution:** a partner with a step grant must read the owner's `review_status_def` label + `review_step_def.is_shared_gate`. Denormalize those onto `review_step` at instantiation rather than opening a cross-org def-table read.

F, System-only (deny-all to users; system role only)

`source_credentials`, `attachment_copy_jobs`, `attachment_refs`, `system_settings`, `credential_access_log`, `failed_ingests`.

Cross-org READ hotspots, resolve, do not leave to break

These handlers read another org's rows today purely because service-role is god-mode. Under user-RLS each returns empty and silently breaks a feature, each needs an explicit resolution: -

handshake.py:get_link_mapping (vendor reads the studio's `source_entity_definitions` + `payload_templates` every time the mapping screen opens). → Redirect to the studio data already frozen in `studio_vendor_links.payload_format_snapshot` at accept time (vendor-readable via the link policy). - **payload.py:get_outbox** (studio's ingestion-status display), confirm it derives status from `payload_field_mappings.ingested_at` (studio-readable via the dispatch FK) rather than needing a read of vendor-owned `payload_export_records`. If it genuinely needs the export rows, add a studio SELECT policy on `payload_export_records` via the dispatch FK chain (`dispatch_id in (select id from payload_dispatches where sender_studio_id in (select current_studio_ids()))`). Decide explicitly, the inventory classified this table dual-party but the draft demoted it to vendor-only. - **reviews.py:create_review** (vendor resolves a studio's `canonical_assets.studio_id`) and **_enrich** (cross-org payload/asset meta) → fold into the relevant RPC / read from the dispatch snapshot, never

the live canonical row. - Agent context builders (`numbersbot` , `lorebot` , `scenario` discussion tools) run as the **user** (\$4), so their reads are correctly RLS-bounded, but verify each returns non-empty for a legitimate caller post-cutover.

Corrections the reviews forced (do not skip)

- **estimate_matrix is owner-private** (findings H3, completeness #4, corrected): studios own base rows (their own internal estimation), vendors own base + per-link override rows; **neither sees the other's**. The constraints `em_one_owner` (exactly one of `studio_id`/`vendor_id`) and `em_link_vendor_only` (`link` $\Rightarrow$ `vendor`) mean override rows carry `studio_id IS NULL` , so a studio's `studio_id` branch can't match them. The studio gets vendor rates **only** through the frozen `estimate_share_dispatches.snapshot` . Do **not** add a studio cross-read of override rows (the rejected `em_studio_link_read / is_link_party` variant), it bypasses the freeze/granularity controls that are the whole point of vendor estimate sharing. **Harden** the studio read branch with `and link_id is null` so the no-leak property does not silently depend on those constraints (defends even if a `studio_id` is ever denormalized onto an override row). Confirm `resolve_effective_matrix` is only ever called in a **vendor** context.
- **studio_join_requests / vendor_join_requests are mixed** (inventory ambiguity): the requester sees their own row (`user_id = (select auth.uid())`), the org admin sees the org's pending rows (`is_org_admin( ... )`). Two SELECT policies. Creation + approval are RPCs (\$7).
- **org_role_audit_log and credential_access_log have NO RLS today**, add it.
`org_role_audit_log` : admin SELECT own org, no user write. `credential_access_log` : pattern F.
- **Drop these malformed policies before creating new ones** (use `DROP POLICY IF EXISTS`):
`asset_reviews` (JWT-claim), `review_attachments` , `scenario_*` ($\times 5$),
`field_bucket_override_log` , `schema_drift_events` , plus the existing JWT-claim policies on every C / B table. Don't migrate them, they're wrong on both axes.

3. Policy shapes (concrete SQL per pattern)

All policies are **command-specific**. RLS already applies to `authenticated` / `anon` / `arthound_system` because they are not table owners, so for the user and system paths, plain `ENABLE ROW LEVEL SECURITY` is sufficient and they are bounded by their policies + grants regardless of FORCE.

FORCE RLS is still applied to every tenant table as defense-in-depth (it makes even the table owner subject to policies, closing the SECURITY DEFINER footgun where `auth.uid()` inside a postgres-owned definer would otherwise misbehave). Generate the FORCE list **dynamically from the catalog**, never a hand-maintained array (ops-finding: a misspelled/renamed table aborts the whole do-block and isn't idempotent):

```

do $$ declare t text; begin
  for t in select tablename from pg_tables
    where schemaname='public' and rowsecurity
      -- EXEMPT the predicate-consulted tables (§1a): owner-run DEFINER helpers must
      -- via the table-owner exemption; FORCE would recurse or silent-deny. review_g
      -- this list when the review subtree ships.
    and tablename <> ALL (array[
      'schema_migrations',
      'studio_members', 'vendor_members', 'studio_vendor_links'
    ])
  loop execute format('alter table public.%I force row level security', t); end loop;
end $$;

```

The exempt tables keep RLS **enabled + policies**, so the user path stays enforced (authenticated is a non-owner); only the owner-run helper reads are exempted. This forces the cross-tenant RPC tables too, which is exactly why the RPC owner-role decision in §6 matters: a FORCE'd table is not writable by a postgres-owned definer without a policy. See §6.

```

-- Membership tables (FORCE-EXEMPT per §1a) - two SELECT policies, writes RPC-only.
-- sm_self is the minimal correct grant and the row resolve_my_membership needs; sm_co backs
-- member-list UI and resolves because current_studio_ids() reads studio_members via owner-e
-- (the table is not FORCE'd). Vendor mirror: vm_self / vm_co with vendor_id / current_vendo
alter table public.studio_members enable row level security;
create policy sm_self on public.studio_members for select to authenticated
    using (user_id = (select auth.uid()));
create policy sm_co on public.studio_members for select to authenticated
    using (studio_id in (select public.current_studio_ids()));
-- (no user INSERT/UPDATE/DELETE - join-approval / role-change / ownership-transfer are RPCs)

-- Pattern B (single-org, studio example) - read own, write own; membership-table writes RPC
alter table public.generated_work enable row level security;
create policy gw_sel on public.generated_work for select to authenticated
    using (studio_id in (select public.current_studio_ids()));
create policy gw_ins on public.generated_work for insert to authenticated
    with check (studio_id in (select public.current_studio_ids()));
create policy gw_upd on public.generated_work for update to authenticated
    using (studio_id in (select public.current_studio_ids()))
    with check (studio_id in (select public.current_studio_ids()));
create policy gw_del on public.generated_work for delete to authenticated
    using (studio_id in (select public.current_studio_ids()));

-- Pattern C (polymorphic) - SELECT-ONLY for users (sec/completeness finding "user can forge
-- data"): replicated_* / source_field_mappings / source_entity_definitions / sync_* are SY
-- truth. NO user INSERT/UPDATE/DELETE - all writes via the system role (§5). Vendor-ingest
-- write goes through the ingest_payload RPC (§6), not a direct user write. Use the INDEX-F
-- disjunction, NOT the scalar is_my_org(owner_type, owner_id): the scalar is a per-row func
-- the planner can't fold into the (owner_type, owner_id) index → Seq Scan on a large replic
-- (ops-finding "per-row blowup"). The disjunction folds each branch to '= ANY(initplan)' an
-- index. Reserve scalar is_my_org() for LOW-row tables (asset_reviews etc.).
create policy ra_sel on public.replicated_assets for select to authenticated
    using ( (owner_type = 'studio' and owner_id in (select public.current_studio_ids()))
    or (owner_type = 'vendor' and owner_id in (select public.current_vendor_ids())) );
-- (no ra_ins / ra_upd / ra_del for authenticated - system role only)

-- generated_work IS a legitimate user write (routes/schedule.py generate_schedule runs in t
-- request) - keep gw_ins/gw_upd/gw_del above (completeness finding: three sections disagree
-- writes win). LoreBot's read of generated_work in a system job needs a system SELECT polic

-- Pattern D (dual-party) - SELECT either party; WRITE owner-only (finding C4).
create policy pd_sel on public.payload_dispatches for select to authenticated
    using ( sender_studio_id in (select public.current_studio_ids())
    or recipient_vendor_id in (select public.current_vendor_ids()) );
create policy pd_ins on public.payload_dispatches for insert to authenticated
    with check (sender_studio_id in (select public.current_studio_ids()));
create policy pd_upd on public.payload_dispatches for update to authenticated
    using (sender_studio_id in (select public.current_studio_ids()))
    with check (sender_studio_id in (select public.current_studio_ids()));
-- Recipient state changes (mark-viewed/ingested) go through RPCs, NOT a recipient UPDATE po
-- payload_data immutability enforced by a BEFORE UPDATE trigger that rejects snapshot edits

-- estimate_matrix is OWNER-PRIVATE. The constraints em_one_owner (num_nonnulls(studio_id, ve
-- and em_link_vendor_only (link_id set ⇒ vendor_id set) mean rows are: studio base (studio
-- vendor base (vendor_id, no link), vendor override (vendor_id + link_id, studio_id NULL).
-- only its OWN studio-owned rows; a vendor sees its own base+override rows. The studio NEVE
-- vendor row - it gets vendor rates ONLY via the frozen estimate_share_dispatches.snapshot
-- granularity the vendor chose. (An earlier draft's em_studio_link_read using is_link_party
-- let a studio read live override rows - a rate-card leak; both reviews flagged it. Removed
-- HARDENING (and link_id is null on the studio branch): override rows have studio_id NULL t
-- studio branch already can't match them - but pinning 'link_id is null' makes the no-leak
-- even if a studio_id were ever denormalized onto an override row, i.e. it does not silentl
-- the ownership constraints. Split into two policies for clarity.
create policy em_studio on public.estimate_matrix for all to authenticated

```

```

using ( studio_id in (select public.current_studio_ids()) and link_id is null )
with check ( studio_id in (select public.current_studio_ids()) and link_id is null );
create policy em_vendor on public.estimate_matrix for all to authenticated
using ( vendor_id in (select public.current_vendor_ids()) )
with check ( vendor_id in (select public.current_vendor_ids()) );

-- Pattern E (review subtree, grant-based)
create policy rv_sel on public.review for select to authenticated
using ( public.is_my_org(owner_org_type, owner_org_id)
        or public.has_grant('review', id, 'view') );

-- Pattern F (deny-all to users): enable RLS, create NO authenticated policy. System policy
alter table public.source_credentials enable row level security;

```

Append-only audit tables (`payload_access_log` , `link_cancellation_audit` , `estimate_share_access_log` , `review_event` , `org_role_audit_log`) get **SELECT** policies for the relevant party and **no user INSERT**, their writes happen inside the cross-tenant RPCs (the `arthound_rpc` -owned `SECURITY DEFINER` fns of \$6, which insert under their own per-table policy in the same transaction as the action they audit). Verify each audit insert lives inside its action's RPC transaction. Watch the `detail` jsonb: both parties can read it, so it must not carry the *other* org's private internals (security finding).

4. Client + token refactor (`lib/db.py`)

The chosen mechanism is a **request-scoped ContextVar** holding the **active token**, set by a FastAPI dependency from `CurrentUser.token`, read by `_headers()`. One shared `httpx` pool stays; only the `Authorization` header varies per call. This avoids threading a `token` param through the ~40 helper modules and the dual-context functions.

```

# lib/db.py
import contextvars, os, httpx

_token_ctx: contextvars.ContextVar[str | None] = contextvars.ContextVar("db_token", default=None)
db_client = httpx.AsyncClient(timeout=30.0, limits=httpx.Limits(max_connections=25, ...))

def _headers(extra: dict | None = None) → dict:
    tok = _token_ctx.get()
    if tok is None:
        # NO silent anon fallback (finding H1). A request/job without a token is a bug - fail
        raise RuntimeError("No DB identity in context - request lacks user token or job lacks anon")
    return {"Authorization": f"Bearer {tok}", "apikey": os.environ["SUPABASE_ANON_KEY"],
            "Content-Type": "application/json", **(extra or {})}

```

- **User path:** a dependency `bind_db_identity(user = Depends(get_current_user))` calls `_token_ctx.set(user.token)`. Add it once at each router (or globally in `main.py`) so every handler runs as the caller. `_user_headers()` is deleted; `_headers()` is the per-identity header builder now.
- **System path:** an async context manager `system_identity()` that `_token_ctx.set(get_system_token())` for the duration of a background job (poll loop, trim loop, attachment-copy worker, webhook sync).

- **Break-glass:** moved to a separate module `lib/db_breakglass.py`, function `service_role_headers()` that `assert os.environ.get("ALLOW_SERVICE_ROLE") == "1"` and is **never** imported by `routes/*`. Grep-able, awkward, audited.
- **Dual-context helpers** (the real blocker, inventory.system): `lib/sync/runner.run_sync`, `lib/canonical.*`, `lib/token_refresh.get_jira_token` run from **both** a user request (login-sync, vendor connector build, payload ingest) and background jobs. They must **not** assume an identity, they read whatever `_token_ctx` holds. The *caller* sets context: a login-triggered sync runs in the user's context; a poll-triggered sync runs inside `system_identity()`. Verify each call site.
- `drain_pages()` keeps defaulting to `_headers()` (now identity-aware), no signature change.

The ~904 `_headers()` / `db_client` call sites do **not** each change; they keep calling `_headers()`, which now returns the contextual identity. The work is (a) adding the two context setters, (b) auditing the dual-context helpers, (c) routing agent endpoints through the **user** context (next point).

Agent endpoints run as the USER, not the system (findings M5 / inventory): `routes/numbersbot.py`, `routes/lorebot.py`, `routes/scenario.py`, `lib/scheduler.py` are user requests. They must run in the caller's context so the AI only ever assembles RLS-permitted rows, otherwise the model becomes a cross-org exfiltration channel. Reserve identity #2 for true background jobs only.

4a. Implementation status (2026-05-30), core landed flag-dormant; route conversions remain

DONE, all gated behind `USE_USER_IDENTITY`; deploys behave byte-for-byte as today until the flip (verified: flag off → service-role; flag on + no token → fail-closed RuntimeError; flag on + token

→ anon+user-token): - `lib/db.py`, `_token_ctx` `ContextVar` + `set/reset/current_token`; `_headers()` branches on `_use_user_identity()` (off → service-role; on → anon + context token, **fail closed**, no silent fallback). `_user_headers` kept only so the auth import doesn't break. -

`lib/system_auth.py`, mints `role=arthound_system` tokens (1h TTL, re-mint at 50%, `jti` logged); `system_identity()` CM; `system_token_accepted()` live probe (startup go/no-go + \$0a deprecation watch). **Signer = ES256 standby key on dev (2026-06-05, \$0a UPDATE):** signs with

`ARTHOUND_SYSTEM_SIGNING_JWK` (private JWK + `kid` header) when set, else legacy HS256/ `SUPABASE_JWT_SECRET` fallback (= rollback). Live on ES256 on BOTH dev (`kid fb73f484-...`) and prod (`kid f456b5dd-...`) as of 2026-06-05. - `lib/db_breakglass.py`, isolated `service_role_headers()` gated by `ALLOW_SERVICE_ROLE=1`, never imported by `routes/*`;

`assert_breakglass_not_in_server()`. - `lib/auth.py`, binds the caller token at the top of `get_current_user` / `_or_pending` (so even the bootstrap read is identity-bound); membership resolves via the `resolve_my_membership()` RPC when the flag is on (DB as source of truth; drops the `app_metadata.role` table-selection). Agent endpoints inherit this binding → run as the user, no per-route change. - `lib/sync/runner.py`, `run_sync` self-opens `system_identity()` inside its lock (single funnel for all triggers). **Design correction (supersedes the §4 "dual-context run_sync" note):** sync is NOT dual-context, `replicated_*` / `sync_*` are system-write-only (§3), so every sync runs as **system**; the route only authorizes *whether* a caller may trigger it. No `as_system` param. - `main.py`, every background loop wraps its DB work in `system_identity()`; `create_task`'d entrypoints (`run_sync`, `_run_generation_task`) self-open; lifespan asserts break-glass-off and runs the system- token probe (blocking when flag on, informational when off) before launching loops.

REMAINING §4 (route-level; each is a flag-ON breakage to fix BEFORE the cutover): 1. **GoTrue Admin API carve-out (§0c):** `members.py:_get_user_emails`, `payload.py:get_outbox`, `_resolve_user`, `user.py delete-account`, `members.py audit email resolution hit` `/auth/v1/admin/users` via `_headers()`. Flag-on returns a `user` token → GoTrue rejects. Add `_admin_headers()` (service-role, admin-API only) and switch these sites. (Or denormalize email into `profiles`.) 2. **Unauthenticated/public reads:** `members.py:resolve_invite_code`, `auth.py:validate_system_invite` / `_get_system_settings`, and `get_current_user_or_pending`'s pending-user **org-name read** (a non-member reading `studios` / `vendors` → denied by `st_sel` / `v_sel` → blank) have no bound token under flag-on → fail closed. Each needs a `SECURITY DEFINER` read-RPC or break-glass. 3. **Signup (§0c):** `auth.py:_signup_create` / `_signup_join` write org+member+join pre-login (no user JWT) and call the GoTrue Admin API anyway → route through `service_role_headers()` or the bootstrap RPCs. 4. **Switch cross-org writes to the new RPCs:** `estimate_share` (`create_estimate_share` + `_log` → `rpc_freeze_estimate_share` / `rpc_revoke` / `rpc_log_view`), `payload ingest` → `rpc_ingest_payload` / `rpc_retry_canonical_link`, `handshake accept/cancel` → `rpc_accept_link_invite` / `rpc_cancel_link`, `members accept/role` → `rpc_decide_join_request` / `rpc_update_member_role`. Live BEFORE the flag flips. 5. **payload_field_mappings ingest-column lockdown** (column grant / BEFORE UPDATE trigger). 6. **Config-UI write path** (`source_field_mappings` / `source_entity_definitions`): user vs system? add scoped user write policies if user-initiated, else the config UI silently breaks. 7. **source_credentials UPDATE-vs-INSERT** + `sync_log` **DELETE-vs-RPC**, finalize migration-2 grants.

§7 (storage byte-gate) and §9 (test matrix) remain as their own steps.

5. System role (identity #2)

```
create role arthound_system nologin;
grant arthound_system to authenticator;           -- PostgREST can SET ROLE to it
-- Least-privilege table grants + policies, NOT god-mode. Ownership stays with postgres so
-- normal RLS applies to arthound_system without needing FORCE RLS (inv_supla: force_rls).
```

Scope matrix (from `inventory.system_role_scope`, the *minimal* set the background jobs touch):

Table	system ops
source_credentials	SELECT, UPDATE (poll read; token-refresh write)
replicated_assets/products/item_types/work	SELECT, INSERT, UPDATE, DELETE (sync)
canonical_assets	SELECT, INSERT, UPDATE
sync_cursors	SELECT, INSERT, UPDATE
sync_log	INSERT, DELETE (trim)
source_field_mappings, source_entity_definitions	SELECT
source_schema_cache	SELECT, INSERT, UPDATE
field_bucket_override_log, schema_drift_events	INSERT
attachment_copy_jobs	SELECT, INSERT, UPDATE, DELETE
attachment_refs	SELECT, INSERT, UPDATE
payload_dispatches	SELECT (attachment-copy worker)
init_jobs	SELECT, INSERT, UPDATE

Each gets a system policy `... to arthound_system using (true) with check (true)` **only for the listed ops**, the GRANT + the narrow op set are the containment, and because it isn't the table owner, RLS still applies (so even a buggy `using (true)` is bounded by the grants). Keep all system grants in **one** migration file with a CI test asserting the live grant set equals this matrix (finding C3-ops: scope drift).

Token lifecycle (findings C1/C2-ops): mint `{role: 'arthound_system', iat, exp}` signed HS256 with `SUPABASE_JWT_SECRET` in `lib/system_auth.py`; **mint eagerly at startup before** spawning `_poll_loop` / `_sync_log_trim_loop`, and **fail fast** (refuse to boot) if minting or the \$0 smoke-test fails, never fall back to service-role/anon. TTL ~1h, re-mint at 50%; **re-check expiry between sync batches** so a long full-sync doesn't die mid-run. Never persist the token. Log every mint with a `jti`. Consider a **separate signing secret** for the system token so anon/JWT rotation and system rotation are independent (finding C2-sec).

6. Cross-tenant RPCs (identity #3), all 13

Owner-role decision (settle this first, the reviews found all four design sections disagreed). The RPCs are `SECURITY DEFINER`, `SET search_path = ''`, `REVOKE EXECUTE FROM public`; `GRANT EXECUTE TO authenticated`. But "owned by postgres bypasses RLS" is **false once \$3's FORCE RLS is on**, a postgres-owned definer on a FORCE'd table is subject to policies and, having none, its writes 42501. Two correct options; **adopt Option A:**

- **Option A (recommended): a dedicated non-owner role `arthound_rpc` owns the RPCs.** `FORCE` only affects the table's *owner*; `arthound_rpc` is a non-owner, so it's subject to RLS normally and needs an explicit policy on **exactly** the tables each RPC writes. This makes the blast radius precisely the cross-write set: even if an in-fn authz check is bypassed, the role cannot touch a table it has no policy/grant on. Where a row constraint is expressible, write it into the policy (`create policy rpc_per on public.payload_export_records for all to arthound_rpc using ( ... ) with check ( ... )`); only where the row legitimately belongs to "the other org" use `to arthound_rpc using (true)` **scoped to that one table**, and then the in-fn authz is load-bearing and must be tested. Never give `arthound_rpc` a blanket `using(true)` across tables.
- Option B (rejected): postgres-owned definer + deliberately **not** `FORCE`-ing the cross-write tables. Simpler but loses defense-in-depth on exactly the most dangerous tables.

The in-function authz check is the real guard regardless: it is load-bearing. Each RPC is one transaction (any `RAISE EXCEPTION` rolls back all writes). For mutating RPCs, `SELECT ... FOR UPDATE` the grant/link rows and re-check inside the lock (TOCTOU). State explicitly what each RPC must **not** do.

Transition rule: flip each route's call to the RPC (and to `_user_headers`) in the *same* deploy that ships the RPC, a DEFINER RPC whose route still passes service-role headers has `auth.uid()=NULL` inside the body → every `is_my_org` check is false → 42501.

Schema accuracy (completeness review caught these against the live DDL, get them right): -

`payload_export_records` columns are `dispatch_id`, `vendor_id`, `vendor_source_type`, `vendor_tool_record_id`, `canonical_asset_id`; the **only** unique constraint is `(dispatch_id, vendor_id)`. The ingest RPC's `INSERT ... ON CONFLICT (dispatch_id, vendor_id) DO NOTHING` must use these names, not `source_type / source_record_id` (those are the `replicated_assets` names). - `payload_access_log` has **actor_studio_id only, no actor_vendor_id**. Add an `actor_vendor_id` column (vendors are actors now) or stash the vendor id in `detail` jsonb; make every RPC audit insert consistent. - Verify `source_credentials` 'encrypted-blob + timestamp' column names before writing the system role's column-scoped UPDATE grant, and confirm token-refresh UPDATES in place vs INSERTs.

Cross-org writes (from inventory.crosswrites): 1. `rpc_ingest_payload(dispatch_id, source_record_id, source_type)`, authz: `dispatch.recipient_vendor_id ∈ current_vendor_ids()` and not revoked/expired. Writes `payload_export_records` + `replicated_assets` stub (owner=vendor) + `payload_field_mappings.ingested_at` + `payload_access_log`. Must **not** touch any row of another vendor or rewrite the studio's canonical asset. 2. `rpc_retry_canonical_link(dispatch_id)`, idempotent re-attempt; same authz. 3. `rpc_accept_link_invite(invite_code)`, authz: the **invite code itself** (no prior cross-org membership). Inserts `studio_vendor_links(active)` with both ids, marks invite accepted, snapshots studio templates onto the link. 4. `rpc_cancel_link(link_id, reason)`, authz: `is_link_party(link_id)`. Sets cancelled, writes `link_cancellation_audit/dispatches`, **cascade-revokes review_grant on the link** (finding C1), optionally revokes live dispatches. 5. `rpc_freeze_estimate_share(link_id, granularity)`, authz: `link.vendor_id ∈ current_vendor_ids()` + active. Projects effective matrix, freezes snapshot, supersedes prior live, logs. **Never** accepts a `studio_id` arg, recipient derived from the link. 6. `rpc_revoke_estimate_share(dispatch_id)`, authz: `series.vendor_id ∈ current_vendor_ids()`. 7.

`rpc_log_estimate_share_view(dispatch_id)` , authz: recipient studio $\in$ `current_studio_ids()`. Append-only. 8. `rpc_approve_join_request(request_id, member_role)` , authz: `is_org_admin` of the request's org. Inserts membership for the **requester's** `user_id`, marks accepted, audits.

Bootstrap RPCs, the #1 fix (completeness #1/#2, finding M4). Without these the app cannot onboard anyone under RLS: 9. `resolve_my_membership()` , returns the caller's (`studio_id`, `vendor_id`, `member_role`) . `get_current_user` calls this instead of a raw `studio_members` `SELECT`, which would otherwise recurse through the membership policy or run with no identity. This is the single most load-bearing change, every authenticated request depends on it. (It's effectively `current*_ids()` + role, in one call.) 10. `rpc_create_studio_with_owner(name)` / `rpc_create_vendor_with_owner(name, handle)` , authz: authenticated + no conflicting membership. Inserts org + caller as owner member. Dissolves the create-org chicken-and-egg. 11. `rpc_request_join(invite_code)` , authz: valid code. Resolves org by code, inserts pending join request for the caller (pre-membership read of an org row they can't otherwise see).

Planned (cross-org-reviews.md): 12. `rpc_grant_review_access(subject_type, subject_id, grantee_link_id, permission)` , authz: caller owns the subject (`is_my_org` on its owner) **and** `is_org_admin` **and** link active. Grantee derived from the link's other party, **never a free-form org id**. Stamps `link_id` on the grant so cancellation can cascade-revoke. 13. `rpc_accept_review_delivery(review_id)` , authz: caller is the accepting org **and** `has_grant(review, id, 'act')` . Validates required shared gates closed, assembles `frozen_snapshot` , sets `accepted_at/by` , writes `review_event` . ACID-critical; lock the review row.

Read-RPC caution (finding H4): any `SECURITY DEFINER` *read* RPC (e.g. resolving studio asset meta for a vendor) bypasses RLS, so it must re-implement the table's RLS internally (confirm the caller is party to the dispatch) and return only the **dispatched snapshot fields**, never the live canonical row. Prefer denormalizing needed fields into the dispatch snapshot at send time (already the pattern) over a live cross-org read RPC.

7. Storage / attachments (the byte layer RLS doesn't cover)

Supabase Storage is a separate service with its own RLS on `storage.objects` ; the table cutover does **not** secure the bucket (findings H6 / E1-ops / completeness #3). `routes/attachments.py` and `routes/reviews.py` stream bytes with service-role storage headers today.

At cutover (mandatory, low-risk): keep byte-serving backend-proxied but add an **RLS metadata gate** , the handler first reads the `review_attachments` / dispatch attachment row **as the user** (RLS applies); if invisible $\rightarrow$ 404 **before any byte fetch**. The byte fetch itself may stay service-role since access was already authorized. `attachment_copy_jobs` / `attachment_refs` stay system-only.

Later (optional): real `storage.objects` RLS policies keyed to the same predicates for direct-from-frontend signed access. Encode owner in the storage path (`/reviews/<review_id>/...` , `/dispatch/<dispatch_id>/...`) now so a future storage policy can match by prefix.

Also audit the **frontend** for any direct `supabase.from()` / `supabase.storage` calls, those will start being RLS-subject (the goal), but someone must confirm the new policies satisfy each one.

STATUS, DONE / VERIFIED 2026-05-30 (dev). The mandatory metadata gate was already structurally present: every byte-serving/upload path performs a user-context (`_headers()`) metadata read before the service-role byte I/O. Gate sites (each now carries an explicit `$7 STORAGE GATE` comment so it can't silently regress to `_admin_headers` / `_storage_headers`): - `attachments.py:get_asset_attachment` → `replicated_assets` read (RLS `ra_sel` , org-scoped). - `attachments.py:_authorize_payload_attachment` (serves `get_payload_attachment`) → `payload_dispatches` read (RLS `pd_sel` , dual-party) + app-layer party/revoked/asset-in-payload checks. - `reviews.py:serve_attachment` → `_fetch_review` (`ar_sel`) + `review_attachments` read (`rat_sel`). - `reviews.py:upload_attachment` (write side) → `_fetch_review` (`ar_sel`) before upload. - `lorebot.py` chat/replicate → `_fetch_vendor_dispatch` (`pd_sel`) before `_read_from_storage` ; blobs are content-addressed, so the dispatch RLS gate (which is where the caller discovers the hash) is the control. `_read_from_storage` documents this contract. All four SELECT policies confirmed live (`ra_sel` / `pd_sel` / `ar_sel` / `rat_sel`). Gate proven live on dev with a minted user JWT: vs service-role ground truth, user A sees 161/201 dispatches, 1/4 reviews, 0/3 review_attachments, 218/247 assets, and an explicit cross-org dispatch read returns `[]` → 404 before any byte fetch. **Frontend audit:** no direct `supabase.storage` / `.from()` data access, all bytes proxied through the backend (the only `getSupabase()` calls are auth/session-token). `storage.objects` RLS (the "Later/optional" direct-signed-access path) intentionally NOT added, nothing reads Storage except the gated backend proxy, so it adds no security today.

8. Sequencing, cutover & rollback

Migration order (each file one transaction; DROP POLICY IF EXISTS ; idempotent): 1. `..._predicates.sql` , predicate fns + `GRANT EXECUTE` . **Guard:** assert the fns exist/are executable before any policy file (finding A3-ops). 2. `..._system_role.sql` , `arthound_system` role + grants + system policies (the scope matrix). 3. `..._policies_*.sql` , drop malformed legacy policies; create the new command-split policies for **all ~50 tables in one pass**. Within each table: **create policies first, enable RLS last**, so any failure leaves the table in its prior working state (finding A2-ops). No `FORCE RLS` for the user path. 4. `..._rpcs.sql` , , all 13 RPCs incl. bootstrap. Additive, old service-role code still works after this lands (finding D1-ops: backward-compatible). 5. **App cutover:** flip the default client from service-role to anon+JWT, route agent endpoints through user context, point `get_current_user` at `resolve_my_membership()` , add the storage metadata gate. 6. `..._drop_breakglass.sql` (**much later**, separate), remove service-role reachability once stable.

STATUS 2026-05-31: step 6 is NOT APPLICABLE on Supabase, by platform constraint. The intended final action was `ALTER ROLE service_role NOBYPASSRLS` . Tested on dev (rolled-back txn): it fails with `42501: "service_role" is a reserved role, only superusers can modify it` . The role we connect as (`postgres`) is `rolsuper=false` on Supabase, and `service_role` is platform-reserved, so its `rolbypassrls` cannot be revoked by us at all. Two consequences: (a) the flag-flip rollback lever is permanent and unremovable, which is fine and arguably desirable; (b) "remove service-role reachability"

is instead satisfied and verified at the APP layer, which is what we control. That guarantee already holds: `_headers()` returns the user/system token under flag-on and never the service key; no `routes/*` module reads service-role for PostgREST (the §9 guard's break-glass-unreachable grep passes); the only service-role uses left are the two sanctioned §0c carve-outs (GoTrue Admin API, Storage bytes). Decision (logged): keep the `_headers()` flag-off service-role fallback in code so a runtime rollback stays possible. The migration is considered COMPLETE without step 6.

The cutover lever is a runtime env flag, not a deploy (findings A1/D2/D3-ops). `USE_USER_IDENTITY` is read **per request**, not at boot, so flip and rollback are independent of code deploys, and **all pods read the same flag value flipped at one instant**, this is what prevents a mixed user-JWT/service-role fleet during a rolling deploy (which would have different pods returning different rows against a FORCE-on DB). If a runtime flag isn't feasible, the cutover deploy must be **atomic** (drain old pods before new pods serve), not rolling. Partial per-route rollback is forbidden: the mixed model is worse than either pure state.

Rollback = flip the flag back to service-role (which bypasses RLS regardless of policy/FORCE). This rests on one fact, **preflight-assert it**: `select rolbypassrls from pg_roles where rolname='service_role'` must be `true` on the target project, or a rollback into FORCE-on tables is deny-all. **service_role stays available as break-glass through the entire window**; only the much-later `...drop_breakglass.sql` (after a 48h soak with zero `NoIdentityError` and a green prod tenancy matrix) removes it.

Stage the system-identity move separately from the user flip. The F-tables already have RLS-on / no-policy *today*, so the instant background loops switch to the system identity, if the system role or `is_system` is broken every loop deny-alls on `source_credentials` and sync/poll/drift/token-refresh all die, **silently**, because every loop body is `except Exception: log.warning ... continue`. So: (a) move loops to the system identity in an **earlier, separate** deploy than the user-default flip, so a failure in either is isolated; (b) make the §0 system-token smoke test a **blocking** go/no-go (mint token, `GET source_credentials` → must return rows; if 403, STOP); (c) **fail-fast at startup**, mint the token and run one live probe *before* launching any loop, raising on non-200 so a misconfigured deploy crashes loudly instead of running with dead sync; (d) escalate `NoIdentityError` specifically to `log.error` + a health metric so a swallowed identity failure can't masquerade as a transient blip; (e) make `system_context()` explicit at the top of every `asyncio.create_task`'d endpoint (`run_sync` , `_run_generation_task` , the attachment workers, drift check, canonical mint) and unit-test each self-opens with no ambient context, `create_task` snapshots context at creation, so relying on inheritance from the loop body is fragile.

Why all-policies-before-flip: a table that is RLS-enabled but policy-less is **deny-all**. Doing every policy in step 3 before the step-5 flip guarantees no authenticated query ever hits an RLS-on-but-policyless table. With no live production data this is a single clean pass, not risky waves.

dev → prod with separate Supabase projects + "never blind db push to prod": apply steps 1-4 to **dev**, run the full test matrix, flip dev's flag, soak. Then apply 1-4 to **prod explicitly** (reviewed `supabase db push` , not blind), run the §9 go/no-go checklist **against prod with a real user JWT**, and only then flip the prod flag. The flag flip and the migration apply are different pipelines , never let the app deploy (flag on) outrun the prod migration (finding A1-ops).

Go/no-go before flipping prod: - [] \$0 minted system token accepted by prod PostgREST (the load-bearing unknown). - [] All policy/fn/system-role/RPC migrations applied to prod; \d+ shows policies on every table. - [] Predicate fns return correct sets for a known test user (run as that user's JWT). - [] \$1a: studio_members / vendor_members / studio_vendor_links are **not** FORCE'd; predicate-fn owner == those tables' owner; zero-membership user → resolve_my_membership empty, no stack-depth. - [] Smoke read as a user JWT returns expected rows (not empty). - [] System token mints at startup; background jobs confirmed running as arthound_system . - [] Attachment serve verified: user-context metadata gate → byte fetch. - [] Audit-log writes (payload_access_log etc.) succeed inside their RPCs under the new identity. - [] Bootstrap flows (signup, create-org, join, approve) work as RPCs. - [] EXPLAIN ANALYZE on replicated_assets + the deepest review subtree shows no per-row predicate blowup. - [] Rollback drill: flip flag → service-role, confirm full recovery. - [] service_role confirmed still present as break-glass (not yet removed).

9. Test matrix (the durable correctness guard)

Build on the existing scripts/test_scoping_fidelity.py . Mint a JWT per **persona** and assert, for every table, the row-visibility contract, run as the **user**, the **system role**, and a **partner-with-grant**:

Persona	Expectation
Studio A user	sees A's rows; zero B's rows; zero unrelated vendor rows
Vendor V user	sees V's rows + dispatches/links it's party to; nothing else
Partner with grant	sees exactly the granted review subtree rows
Partner without grant	sees nothing of that subtree
Multi-org user	sees the union of their orgs (finding H2, every consumer handles the set)
arthound_system	sees only the scope-matrix tables/ops; not god-mode
Anon (no JWT)	deny-all everywhere except explicit login endpoints
Break-glass	not reachable from any routes/* import (static grep test)

Run via pytest hitting PostgREST with minted per-persona JWTs (and pgTAP for in-DB policy assertions if desired). **Highest-regression-risk policies to test first:** 1. estimate_matrix **no studio cross-read**, seed a vendor base row, a vendor **override** row (vendor_id + link_id , studio_id NULL) on a link whose studio is persona A, and a studio-A base row. Assert studio A reads **only its own base row**, zero vendor base rows, **zero override rows**. This is the test the rejected em_studio_link_read policy would have *failed* and the corrected split policy passes; assert it against the override row specifically. 2. \$1a **recursion / silent-deny**: (a) a zero-membership user calls resolve_my_membership → empty, **no stack-depth error**; (b) FORCE-set regression: studio_members , vendor_members , studio_vendor_links are **not** in pg_class.relforcerowsecurity ; (c) preflight: relowner of each exempt table == proowner of the predicate fns. 3. payload_dispatches recipient-can't-write (write goes through the RPC, not a recipient UPDATE policy). 4. has_grant link-liveness + post-cancellation revocation (gated to the

review-subtree phase). 5. the polymorphic `is_my_org` type-binding (studio id can't match a vendor-owned row and vice versa).

This matrix is what lets you *trust* RLS instead of hoping, keep it green in CI.

STATUS, DONE 2026-05-30 (dev): `scripts/rls_persona_matrix.py`. A standalone, CI-runnable guard (repo convention, no pytest infra) that mints a JWT per persona and hits PostgREST directly, so it exercises the live policies regardless of `USE_USER_IDENTITY` (the flag only changes which header the *app* sends; the guard always sends a user/anon/system bearer). Personas are DISCOVERED from live data (a real studio member + vendor member), so it runs on dev or prod with no hardcoded fixtures; cases with no data report SKIP, not FAIL. Default run is READ-ONLY (safe against prod); `RLS_MATRIX_SEED=1` opts in to a transient override-row seed (always cleaned up in `finally`) so R1 is exercised even on an override-free DB. Coverage: studio/vendor own-only visibility (`replicated_assets`, `asset_reviews`, `payload_dispatches`, `estimate_matrix`); R1 `estimate_matrix` no-studio-cross-read (base-only + seeded override-hidden); R3 recipient-can't-write; R5 `is_my_org` type-binding; F-table + anon deny-all; system identity (#2) reads the system-scoped F-table a user can't; break-glass-unreachable static grep over `routes/*`. R2a (zero-membership → no stack-depth) is in the guard; R2b/c (FORCE-set + fn-owner) is a catalog assertion not reachable over PostgREST, the guard SKIPS it with the companion SQL in its footer (both confirmed live: the 3 membership tables `relforcerowsecurity=false`; predicate-fn owner == membership-table owner == postgres). Result on dev: **15 PASS / 0 FAIL / 3 SKIP** (seeded), the SKIPS are data-shape (no 2nd studio with assets / no multi-org user / catalog R2bc). Multi-org union (H2) asserts when such a user exists. Keep this green before every flag flip.

10. Performance notes

- Predicate hoisting: `in (select fn())` → `InitPlan`, evaluated once per statement. Verify on `replicated_assets` at scale; fall back to a per-request `SET LOCAL app.studio_ids` GUC if needed.
 - `review_grant` indexes: `(subject_type, subject_id) WHERE revoked_at IS NULL and (grantee_org_type, grantee_org_id, subject_type, subject_id) WHERE revoked_at IS NULL` (findings B1/B2-ops). Plain `CREATE INDEX` (transactional) is fine at current scale; reserve `CREATE INDEX CONCURRENTLY` (own migration file, no other DDL) for when tables grow (finding F2-ops).
 - `resolve_visible_review` demotes from security boundary to a one-pass read aggregator with RLS underneath; confirm grant-based visibility composes across the review→step→comment join with a worked example + test (completeness #8).
 - The 15s membership cache in `lib/auth.py` is app-side only and doesn't affect DB predicates (which read live). App and DB can disagree for ≤15s on role gating only, acceptable; document it.
-

11. Residual risks & open decisions

- **Asymmetric JWT mode (\$0)**: the load-bearing unknown; resolve before building.

- **estimate_matrix override exposure:** confirm whether studios should ever read raw override rows directly, or only via the frozen share snapshot (recommend snapshot-only).
- **Multi-org users:** RPCs that assume a single acting org must take the acting `org_id` as an arg and verify membership, never pick the first (finding H2).
- **TOCTOU on cross-org RPCs:** lock grant/link rows in mutating RPCs (finding M2).
- **Separate system signing secret:** adopt for blast-radius isolation (finding C2-sec).
- `trim_sync_log()` and other existing RPCs, review whether they're `SECURITY DEFINER` and which role they need under the new model.

12. ArtHound-principle check

- **Security/integrity first:** the boundary moves into Postgres; the inline filters you already have invert from sole-boundary to a true second layer ("defense in depth" finally earns the name).
- **No cross-org flow without explicit authorization:** every cross-org write is a named, audited `SECURITY DEFINER` RPC with a load-bearing in-function authz check; every grant is a row + an event.
- **service_role = migrations + break-glass only:** removed from request paths, awkward to reach, grep-able, gated by an env assertion.